

SQLite with Node.js

SQLite is a lightweight, file-based database that's great for local storage. It doesn't require a server, so it's easy to use for small to medium-sized applications.

To work with SQLite in Node.js, we'll use the `sqlite3` package, which provides an easy-to-use interface for interacting with SQLite databases.

1. Installation

First, you'll need to install the `sqlite3` package. You can do this using npm:

```
npm install sqlite3
```

2. Creating a Database and Table

Here's a basic example that shows how to create a SQLite database, create a table, and insert data into it:

```
const sqlite3 = require('sqlite3').verbose();

// Open the database (it will create a new database file if it doesn't exist)
const db = new sqlite3.Database('myDatabase.db', (err) => {
  if (err) {
    console.error('Error opening database', err);
  } else {
    console.log('Connected to SQLite database');
  }
});

// Create a table if it doesn't exist
db.run(`CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT,
  email TEXT
)`, (err) => {
  if (err) {
    console.error('Error creating table', err);
  } else {
    console.log('Table created or already exists');
  }
});

// Insert some data
```

```
const insertQuery = `INSERT INTO users (name, email) VALUES (?, ?)`;

db.run(insertQuery, ['John Doe', 'johndoe@example.com'], (err) => {

  if (err) {

    console.error('Error inserting data', err);

  } else {

    console.log('Data inserted successfully');

  }

});
```

```
// Close the database connection

db.close((err) => {

  if (err) {

    console.error('Error closing the database', err);

  } else {

    console.log('Database connection closed');

  }

});
```

3. Performing CRUD Operations

You can also perform CRUD (Create, Read, Update, Delete) operations with SQLite.

Read Data

To read data, you can use the `all()` or `get()` methods to fetch results.

// Reading data from the 'users' table

```
const getUsers = () => {

  db.all('SELECT * FROM users', (err, rows) => {

    if (err) {

      console.error('Error reading data', err);

    } else {

      console.log('Users:', rows);

    }

  });

};

getUsers();
```

Update Data

To update data, use the `run()` method with an update query.

```
const updateUser = (userId, newName) => {
  const updateQuery = `UPDATE users SET name = ? WHERE id = ?`;
  db.run(updateQuery, [newName, userId], (err) => {
    if (err) {
      console.error('Error updating data', err);
    } else {
      console.log('User updated successfully');
    }
  });
};
```

```
updateUser(1, 'Jane Doe'); // Update user with ID 1 to "Jane Doe"
```

Delete Data

To delete data, use the `run()` method with a delete query.

```
const deleteUser = (userId) => {
  const deleteQuery = `DELETE FROM users WHERE id = ?`;
  db.run(deleteQuery, [userId], (err) => {
    if (err) {
      console.error('Error deleting data', err);
    } else {
      console.log('User deleted successfully');
    }
  });
};
```

```
deleteUser(1); // Delete user with ID 1
```

4. Working with SQLite in Async Mode

SQLite can be synchronous or asynchronous. By default, it is synchronous. However, it's generally a good practice to use it in asynchronous mode to avoid blocking the event loop in a Node.js application.

Here's an example of how you can use the asynchronous pattern:

```
const sqlite3 = require('sqlite3').verbose();
const db = new sqlite3.Database('myDatabase.db');
```

```
// Asynchronous query example
```

```
db.all('SELECT * FROM users', (err, rows) => {  
  if (err) {  
    console.error('Error fetching data', err);  
  } else {  
    console.log('Users:', rows);  
  }  
});
```

5. Closing the Database Connection

Always remember to close the SQLite connection when you're done.

```
db.close((err) => {  
  if (err) {  
    console.error('Error closing the database', err);  
  } else {  
    console.log('Database connection closed');  
  }  
});
```

Conclusion

With SQLite, you can easily manage your application's data using a lightweight, file-based database. It's a great choice for small applications or as a local storage solution in mobile apps, IoT devices, or Electron apps.

Question 1: .verbose() in SQLite

The `.verbose()` method in SQLite is used to enable verbose mode. This method is a part of the `sqlite3` module and makes SQLite log detailed error messages and warnings to the console.

When you call `sqlite3.verbose()`, it helps in debugging by printing detailed logs about query execution and any issues that may arise. This can be useful when you're troubleshooting or trying to track down issues with your queries.

```
const sqlite3 = require('sqlite3').verbose(); // This enables verbose mode
```

Without `.verbose()`, SQLite may not show detailed error messages or debug information, which can make it harder to troubleshoot issues.

Question 2: What is the code difference between synchronous and asynchronous operations in SQLite?

- **Synchronous Example (Default Behavior):**

In the synchronous (blocking) mode, SQLite operations are blocking, meaning the execution of the code will wait for the query to complete before moving to the next line of code.

```
const sqlite3 = require('sqlite3').verbose();  
const db = new sqlite3.Database('myDatabase.db');
```

```
// Blocking (synchronous) read  
db.all('SELECT * FROM users', (err, rows) => {  
  if (err) {  
    console.error('Error reading data', err);  
  } else {  
    console.log('Users:', rows);  
  }  
});
```

```
console.log('This line is executed after the database query');
```

Here, the program will wait until the `db.all()` method is done before moving to the next line of code.

- **Asynchronous Example (Non-blocking):**

In asynchronous (non-blocking) mode, SQLite operations are handled in the background, and the rest of the code can continue executing without waiting for the database operation to complete.

```
const sqlite3 = require('sqlite3').verbose();  
const db = new sqlite3.Database('myDatabase.db');
```

```
// Non-blocking (asynchronous) read  
db.all('SELECT * FROM users', (err, rows) => {
```

```
if (err) {  
  console.error('Error reading data', err);  
} else {  
  console.log('Users:', rows);  
}  
});
```

```
console.log('This line is executed immediately after the database query');
```

Here, the `console.log('This line is executed immediately after the database query')` is executed while the database query is still running, which makes the process non-blocking.

Question 3: Is there something like Mongoose for SQLite?

No, there isn't something exactly like **Mongoose** for SQLite. Mongoose is an Object Data Modeling (ODM) library designed specifically for MongoDB. It provides higher-level abstraction and additional features such as schema validation, model relationships, middleware, and more.

For SQLite, you don't have a similar ODM, but you can still manage your database with SQL queries directly or use libraries like **Sequelize** (which is a promise-based ORM for SQL databases, including SQLite) to add some abstraction to your database operations.

Here is an example using Sequelize with SQLite:

```
const { Sequelize, DataTypes } = require('sequelize');  
  
// Initialize Sequelize with SQLite  
const sequelize = new Sequelize({  
  dialect: 'sqlite',  
  storage: 'myDatabase.db',  
});  
  
// Define a model (similar to Mongoose schema)  
const User = sequelize.define('User', {  
  name: {  
    type: DataTypes.STRING,  
    allowNull: false,  
  },  
  email: {  
    type: DataTypes.STRING,  
    unique: true,
```

```
},  
});  
  
// Sync models with the database  
sequelize.sync()  
  .then(() => {  
    console.log('Database synced!');  
  })  
  .catch((err) => {  
    console.error('Error syncing database:', err);  
  });
```

Here, **Sequelize** provides an abstraction for SQLite (and other SQL databases) and helps manage your database with models, validation, associations, and more.

Question 4: What should be the proper folder structure for a large project with multiple collections like student, teacher, course, exam, etc.?

For a large Node.js project that uses SQLite and manages multiple collections (like students, teachers, courses, exams), you should follow a modular structure. Each module (collection) can have its own folder with models, routes, controllers, and services.

Here's a suggested folder structure:

```
/project-root  
/  
  controllers  
    studentController.js  
    teacherController.js  
    courseController.js  
    examController.js  
  models  
    studentModel.js  
    teacherModel.js  
    courseModel.js  
    examModel.js
```

/routes

studentRoutes.js

teacherRoutes.js

courseRoutes.js

examRoutes.js

/services

studentService.js

teacherService.js

courseService.js

examService.js

/utils

db.js # SQLite connection setup

/config

config.js # Database configuration and other constants

/middlewares

authMiddleware.js

/views

index.pug # Or EJS, if you use templating engine

/public

/images

/css

/js

server.js # Entry point to the application

Explanation:

- **controllers/**: Handles the logic for various routes (e.g., user operations like adding a student).
- **models/**: Defines the structure of your collections (students, teachers, courses, etc.).
- **routes/**: Contains the Express routes for different collections (e.g., routes for students, teachers, etc.).
- **services/**: Business logic related to your data (e.g., handling complex operations for students).
- **utils/**: Contains utility functions, like setting up database connections.
- **config/**: Stores configuration variables like database connection strings.

- **middlewares/**: For authentication or any other middleware logic.
- **views/**: Views/templates for the app (if you are rendering HTML pages).
-

Question 5: How to use multiple databases, and each have multiple collections?

To use multiple databases (like acs and dcs), you can create separate connections for each database. You would then use these connections to interact with the collections in each respective database.

Here is an example showing how to use multiple SQLite databases and work with multiple collections:

```
const sqlite3 = require('sqlite3').verbose();

// Create separate connections for different databases

const acsDb = new sqlite3.Database('acs.db'); // ACS Database
const dcsDb = new sqlite3.Database('dcs.db'); // DCS Database


// ACS Database operations
acsDb.all('SELECT * FROM students', (err, rows) => {
  if (err) {
    console.error('Error fetching ACS students', err);
  } else {
    console.log('ACS Students:', rows);
  }
});


// DCS Database operations
dcsDb.all('SELECT * FROM teachers', (err, rows) => {
  if (err) {
    console.error('Error fetching DCS teachers', err);
  } else {
    console.log('DCS Teachers:', rows);
  }
});
```

```
// Close the databases
```

```
acsDb.close();
```

```
dcsDb.close();
```

1. What You Have Right Now:

You're putting the database logic directly into a **model file**, like:

js

CopyEdit

```
// models/userModel.js
```

```
const createUser = (username, email, password, callback) => {  
  const hashedPassword = bcrypt.hashSync(password, 10);  
  db.run(  
    `INSERT INTO users (username, email, password) VALUES (?, ?, ?)`,  
    [username, email, hashedPassword],  
    function (err) {  
      callback(err, this.lastID);  
    }  
  );  
};
```

Question: Can this be done in the controller?

Yes, but **should it be**? Not recommended.

The **controller** should not contain raw SQL or heavy business logic like hashing passwords, connecting to DB, or querying directly.

Why Model is the Right Place:

✓ Separation of Concerns:

- **Model** → talks to the database (queries, CRUD)
- **Controller** → handles user input and responses (HTTP)
- Keeps your code **modular, clean, reusable**

✓ Testability:

- Models are easier to test without needing to run the full app or server.

✓ Scalability:

- Easier to maintain when your app grows (you don't want SQL code in every route).
-

🔗 Controller Should Look Like This:

js

CopyEdit

```
// controllers/userController.js
```

```
const { createUser, findUserByEmail } = require('../models/userModel');
```

```
const handleRegister = (req, res) => {  
  const { username, email, password } = req.body;
```

```
  createUser(username, email, password, (err, userId) => {  
    if (err) {  
      return res.status(500).json({ error: 'User registration failed' });  
    }  
    res.status(201).json({ message: 'User created', userId });  
  });  
};
```

💻 If You Write SQL in Controller

js

CopyEdit

```
// ❌ Not recommended
```

```
db.run(`INSERT INTO users (...)`, ...);
```

You tightly couple logic with route handling. When the project scales, things will break down:

- Difficult to refactor
 - Hard to reuse logic
 - Harder to test or debug
-

✅ Best Practice Summary

Layer	Responsibility	Should contain
Model	Business logic, database queries	SQL queries, data manipulation
Controller	Route logic, input validation, response	Calls model functions, handles status codes

1. db.run() → For *writing* (INSERT, UPDATE, DELETE)

Use when:

- You want to **insert**, **update**, or **delete** data.
- It **does not return** any row results (but can return lastID or changes count).

Example:

js

CopyEdit

```
db.run(
  `INSERT INTO users (username, email, password) VALUES (?, ?, ?)`,
  [username, email, hashedPassword],
  function (err) {
    if (err) return console.error(err.message);
    console.log('Inserted row with ID:', this.lastID); // <- Available here
  }
);
```

2. db.get() → For *fetching ONE row*

Use when:

- You need to **get a single record** (e.g., a user by email or ID).
- It returns **only one row**.

Example:

js

CopyEdit

```
db.get('SELECT * FROM users WHERE email = ?', [email], (err, row) => {
```

```
if (err) return console.error(err.message);

console.log('User:', row); // <- only one user object
});
```

3. db.all() → For *fetching ALL matching rows*

Use when:

- You want to fetch **multiple rows** (like a full list).
- It returns an **array of rows**.

Example:

```
js
CopyEdit
db.all('SELECT * FROM users', (err, rows) => {
  if (err) return console.error(err.message);
  console.log('All users:', rows); // <- array of user objects
});
```

Summary Table

Function	Use Case	Returns	Example Query
db.run()	INSERT / UPDATE / DELETE	No rows (just info)	INSERT INTO, DELETE FROM, UPDATE
db.get()	SELECT single row	One row (object)	SELECT * FROM users WHERE id = ?
db.all()	SELECT multiple rows	Array of rows	SELECT * FROM users

If you're building a CRUD app, you'll commonly use:

- db.run() for **Create, Update, Delete**
- db.get() for **Read one**
- db.all() for **Read many**

Can we use .all() for all types of queries?

Short answer:  No, .all() is only meant for **SELECT** queries that return **multiple rows**.

🌟 Why?

The `.all()` method is designed to:

- Execute a SELECT query
 - Return **an array of all matching rows**
 - It **doesn't work properly with** INSERT, UPDATE, or DELETE, because those operations:
 - **Don't return rows**
 - Just return info like number of rows affected (`this.changes`) or last inserted ID (`this.lastID`)
-
-
-

✅ In MongoDB (with Mongoose), we don't have to manually write such basic queries because Mongoose gives us built-in methods based on the schema.

📄 Comparison: SQLite (manual) vs MongoDB (automatic)

♦ SQLite with sqlite3

You have to manually:

- Write the query (INSERT, SELECT, etc.)
- Pass parameters
- Handle callbacks
- Hash passwords before saving

js

CopyEdit

```
db.run('INSERT INTO users (...)', [...], (err) => {});
```

```
db.get('SELECT * FROM users WHERE email = ?', [...], (err, row) => {});
```

♦ MongoDB with Mongoose

Mongoose gives you model-based methods out of the box:

js

CopyEdit

```
const user = new User({ username, email, password });
```

`await user.save(); // ✅ createUser automatically`

`const user = await User.findOne({ email }); // ✅ findUserByEmail automatically`

No need to manually write:

- SQL statements
- Callback handlers (use `async/await`)
- `module.exports` for each operation (you can export model once)

Example: Mongoose User Model

js

CopyEdit

// models/User.js

```
const mongoose = require('mongoose');
```

```
const bcrypt = require('bcrypt');
```

```
const userSchema = new mongoose.Schema({
```

```
  username: String,
```

```
  email: { type: String, unique: true },
```

```
  password: String,
```

```
});
```

```
// Password hashing middleware
```

```
userSchema.pre('save', async function (next) {
```

```
  if (this.isModified('password')) {
```

```
    this.password = await bcrypt.hash(this.password, 10);
```

```
  }
```

```
  next();
```

```
});
```

```
const User = mongoose.model('User', userSchema);
```

```
module.exports = User;
```

 Usage:

js

CopyEdit

// Register

```
const newUser = await User.create({ username, email, password });
```

// Login

```
const user = await User.findOne({ email });
```

Summary:

Feature	SQLite (sqlite3)	MongoDB (mongoose)
Schema-based queries	✗ No	✓ Yes
Auto hashing/passwords	✗ Manual	✓ Middleware (pre('save'))
Create/Find functions	✗ Manual	✓ Built-in (.create(), .findOne() etc.)
Async/await	✗ Not by default	✓ Yes